Project: Electric Vehicle Charging Station Locator
Version: 1
Status: Core CLI workflow implemented; local end-to-end and live-data verification still needed.

1. Project Overview

The EV Charging Station Locator is all about helping EV drivers to find a charging station in the area and filtering results to show only those that meet their criteria. The original concept was a mobile app with an interactive map. For v1, we've kept it simple and focused on the data flow using a Python CLI rather than a full graphical interface.

2. What is implemented so far

The current main.py includes these parts of the Version 1 workflow:

• A menu to start a search or to exit.

• Lat/long input.

• A request to the ArcGIS Alternative Fueling Stations feature service for electric stations within a 25-mile search radius.

• Conversion of the returned ArcGIS attributes into a simpler station structure.

• Approximate straight-line distance using the Haversine formula.

• Optional filters for maximum distance, connector type, minimum charging power, maximum price, and availability/status.

• A results list sorted by distance, station selection, and a station details view.

• A local JSON sample data fallback in the event of a live request failure.

• Project support files (README, bug sheet, sketch, specification and license)

The application is currently CLI-only. An interactive map, directions, and a graphical/web interface are not implemented in Version 1.

3. Demonstrating and testing status

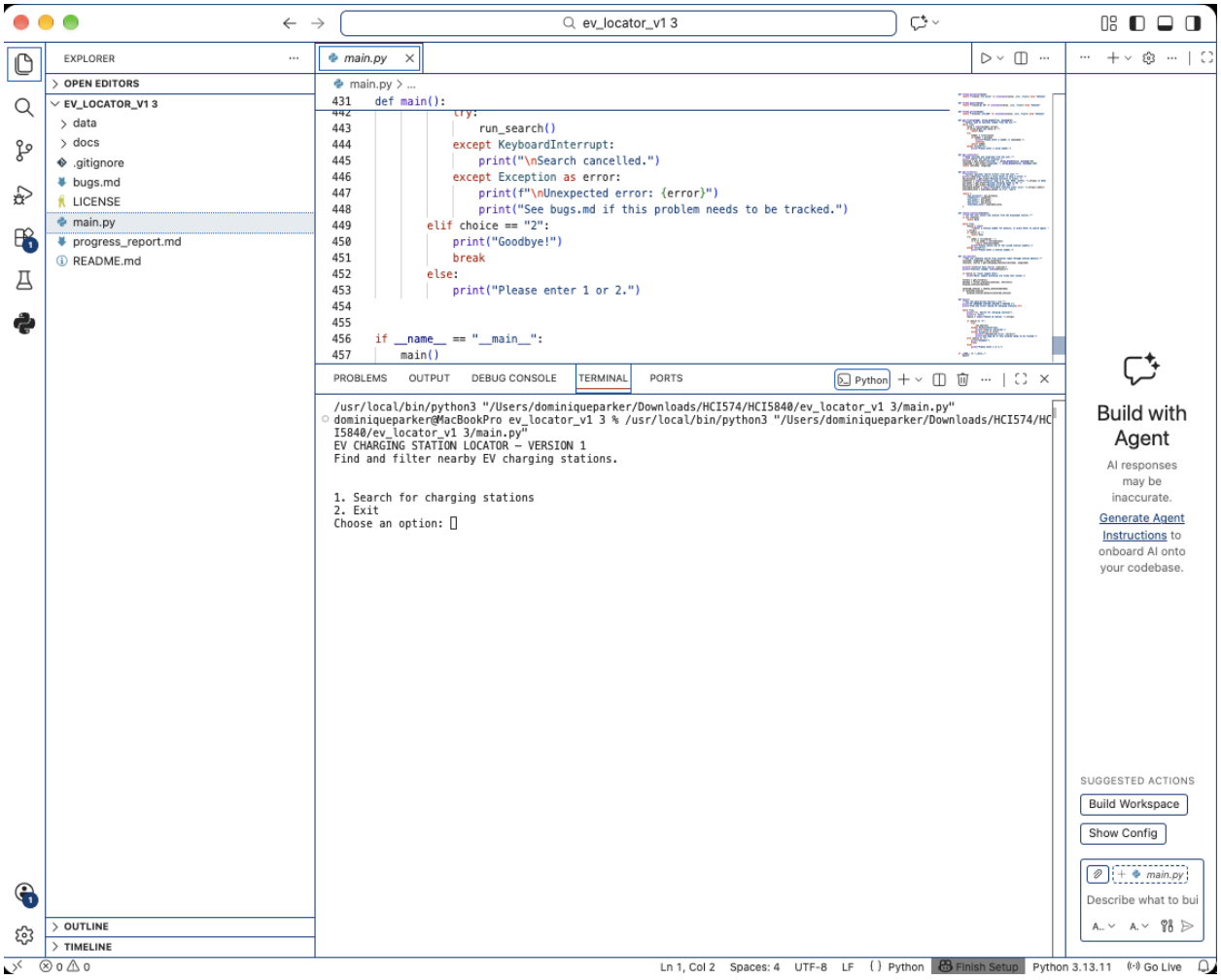


Figure 1: **EV Charging Station Locator Main Menu**: The command-line application launches successfully and presents the user with options to search for nearby EV charging stations or exit the program.

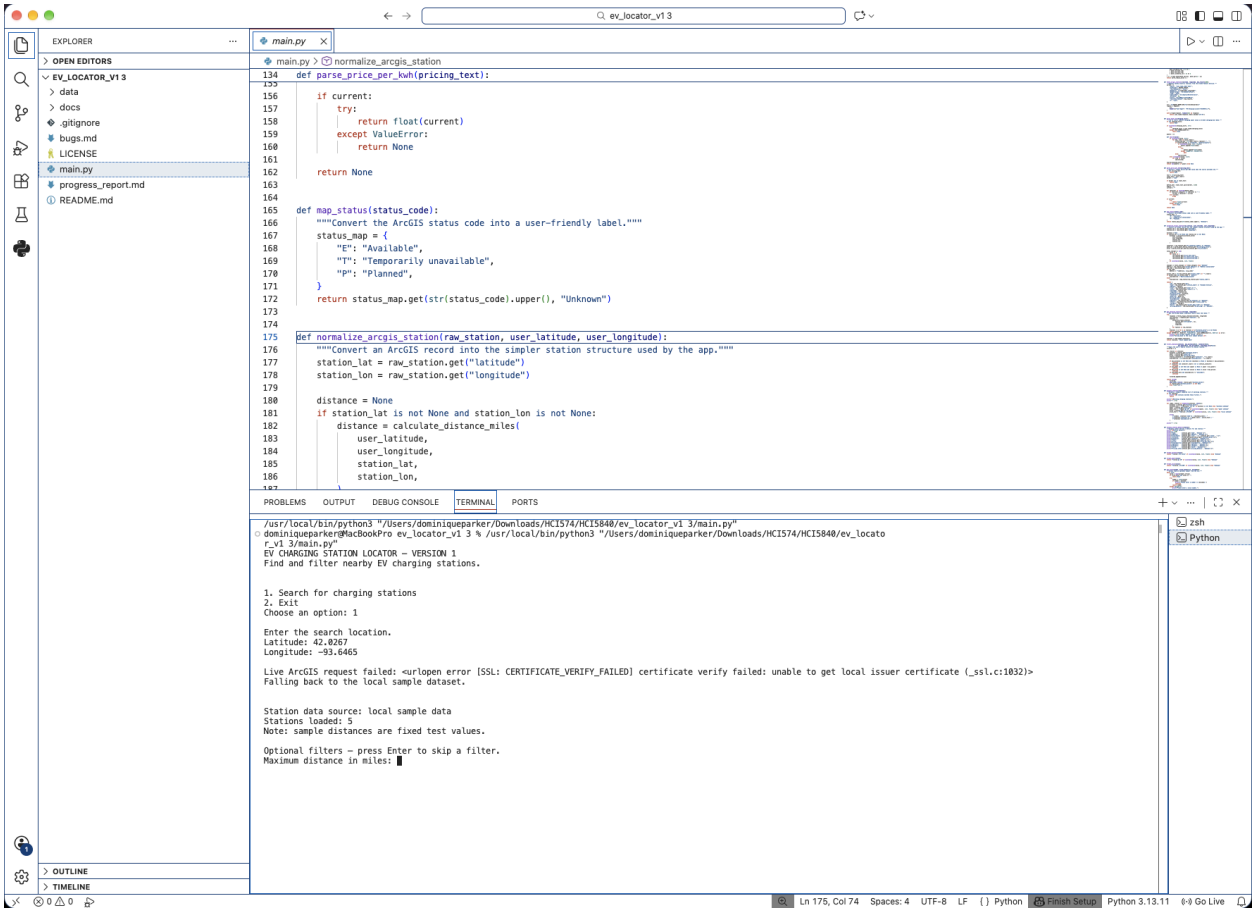


Figure 2: **Search workflow and ArcGIS connection error.** After entering the search coordinates, the application attempts to retrieve charging station data from ArcGIS. The request fails because of an SSL certificate verification error, so the application falls back to the local sample dataset and loads five stations. This demonstrates that the fallback mechanism works, although live data retrieval still needs to be resolved.

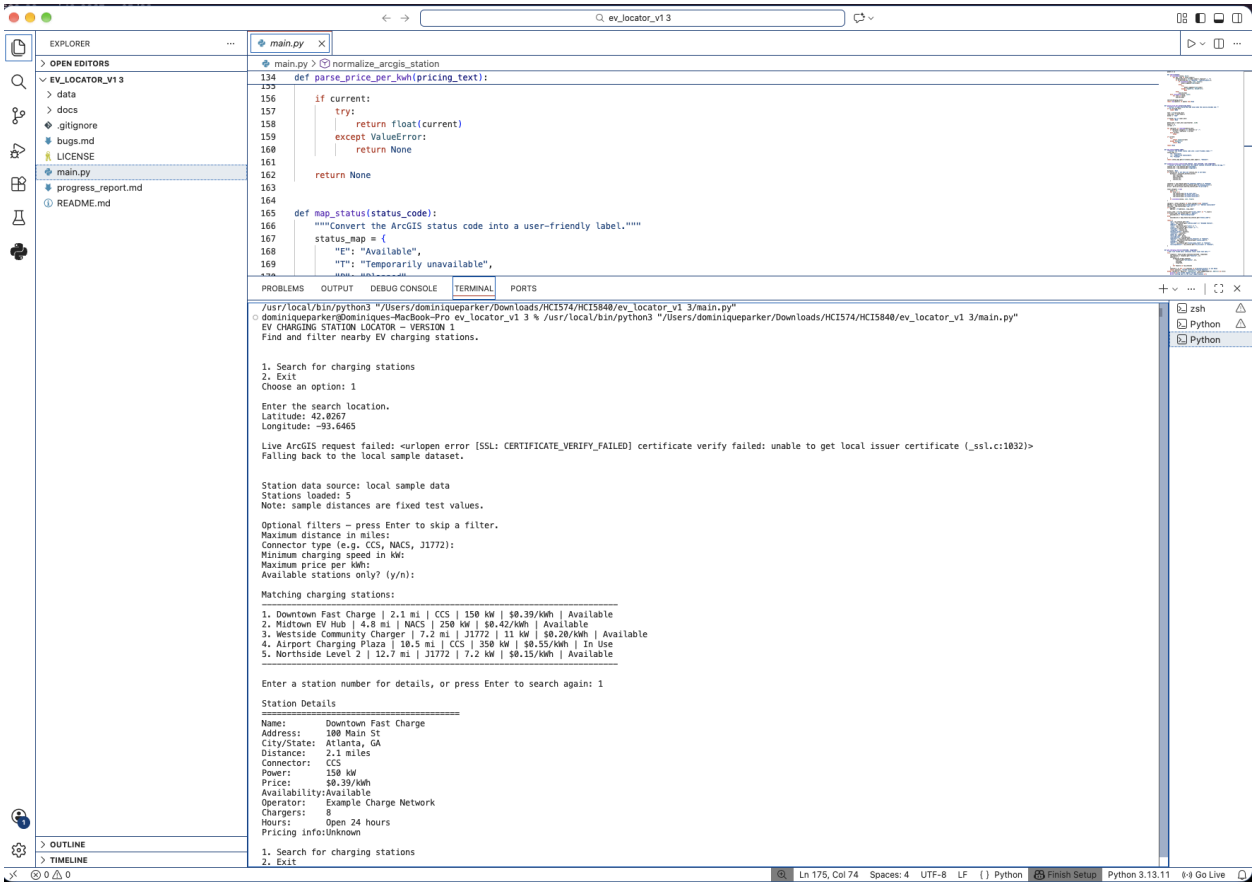


Figure 3: **Charging Station details view:** After selecting the first result, the application displays detailed information about the selected charging station, including its address, distance, connector type, charging power, availability, operator, number of chargers, and operating hours. This demonstrates that users can move from the search results to an individual station's details. The information shown comes from the local sample dataset because the live ArcGIS request is currently failing.

The code has been reviewed for its intended control flow, and the project includes a sample-data fallback. However, a successful live ArcGIS request and a complete local end-to-end run have not been confirmed yet.

4. Issues and Limitations

• **Live data and network dependency:** The ArcGIS request is reliant on internet access and will fail. The code will fall back to local sample data in the event of certain request or parsing errors. Next step: test both the live path and the fallback path locally.

• **Missing price or charging-power values:** The source does not always provide a consistently parseable numeric price-per-kWh or power value. When values are missing, the app displays `Unknown`, and the related filters exclude those stations. Next step: keep these fields optional and make the behavior clear to users rather than assuming the source has complete data.

• **Availability accuracy:** The dataset's status/access fields do not guarantee real-time charger availability. The app should refer to this as source-reported status/access information rather than as a live guarantee that a charger is free.

• **Location input:** Version 1 requires latitude and longitude rather than a city, an address, or an auto-determined device location. This is a deliberate scope reduction for the CLI.

• **Result coverage:** The request caps results at 100 records. The initial search radius is 25 miles; a user-entered maximum distance filter can be used to narrow the results, but it won't retrieve any stations outside of the API query radius.

5. Where help is needed

The key area where instructor feedback would be useful is in validating the ArcGIS query and the assumptions made in the field normalization. Specifically, I'd like feedback on whether the current approach to dealing with missing price, power, and status is appropriate for a V1 prototype. If the live service isn't returning the fields we expect consistently, then I'd stick with the CLI and adjust the data handling rather than go down the route of a GUI at this stage.

6. Self reflection

I think the biggest change so far has been reducing the original mobile/map idea to a CLI. When I was first brainstorming ideas, I was looking at it from a UX designer's perspective and had to remember that this is a programming project, not a design one. I was extremely overwhelmed at first, but rescaling it has made me feel a little better. This change makes the project more manageable and lets me focus on the part that supports the rest of the application: retrieving, processing, and filtering station data. The implementation has helped turn the original idea into a concrete workflow, but the external data source is still the

biggest uncertainty. I am cautiously optimistic about completing a useful prototype if I prioritize verification and reliability over adding a graphical interface. The most difficult parts, I believe, will be working with API responses and handling missing or inconsistent fields. A key takeaway is that having a data source does not automatically mean it provides every field in the format the application needs. For the next stage, I need to test the complete workflow and use the results to decide what is realistic to promise